Матрица проверок

Проверка	Что доказывает	Команда	CI
Тесты backend на SQLite	логика не требует внешней СУБД	`make test`	`ci.yml` → `backend`
ruff + mypy	статические правила соблюдены	`make lint typecheck`	`ci.yml` → `backend`
Модели ↔ миграции согласованы	схема не расходится с кодом	`make alembic-check`	`ci.yml` → `backend`
Сборка фронтенда	UI собирается воспроизводимо	`make build-frontend`	`ci.yml` → `frontend`
Миграции на пустой БД	установка с нуля возможна	`make alembic-upgrade`	`portability.yml` → `alembic-fresh`
Повторный `upgrade head`	обновление идемпотентно	`make alembic-upgrade` дважды	`portability.yml` → `alembic-upgrade-path`
Backup → restore → smoke	данные восстанавливаются	`pg_dump` → `pg_restore` → `make smoke-test`	`portability.yml` → `backup-restore`
Полный стек поднимается	сервисы взаимодействуют	`make compose-up` + `make smoke-test`	`portability.yml` → `compose-smoke`
Запуск без интеграций	GigaChat/Keycloak/S3 — опции	`make compose-up SERVICES="backend frontend"`	`portability.yml` → `config-without-secrets`
Документация согласована	ссылки живые, спецификации модулей полны	`make docs-check`	`portability.yml` → `docs`
Проверка хоста	окружение пригодно до запуска	`make doctor`	локально
Проверки запускаются на каждой ветке	работа в ветке не доживает до merge непроверенной	`push` в любую ветку + PR	`ci.yml`, `portability.yml` (`branches: ["**"]`)
Зеркало CI не разошлось с GitHub	результат не зависит от площадки	сравнение шагов `.sourcecraft/ci.yaml` с `ci.yml`	вручную при изменении CI

Что не проверяется автоматически

- Реальный обмен с LMS/CMS: адаптер mock, контракт проверяется dry-run-тестами.
- Производительность и нагрузка.
- Прод-контур Yandex Cloud: проверяется вручную по operations/deployment.md.
- Соответствие двух описаний CI (`.github/workflows/ci.yml` и `.sourcecraft/ci.yaml`) по набору шагов — расхождение возможно, см. аудит.